

Core Java Fundamentals

Basic Level Technical Interview Guide & Runtime Mechanics (50+ Questions)

Coverage: Fundamentals, OOP, Strings, Exceptions, Memory, Java 8	Target: Core Java Concepts & Interview Essentials	Format: Print-Ready PDF
---	--	--------------------------------

1. Java Fundamentals (Q1 - Q10)

Q1 What is Java? What are its main features?

Java is a high-level, class-based, object-oriented programming language designed by Sun Microsystems (Oracle).

Main Features:

- **Platform-Independent (WORA):** "Write Once, Run Anywhere" via Bytecode & JVM.
- **Object-Oriented:** Models software around classes, objects, and OOP pillars.
- **Robust & Secure:** Automatic memory management (GC), strong type checking, and lack of explicit pointers.
- **Multithreaded:** Built-in support for concurrent thread execution.
- **High Performance:** Just-In-Time (JIT) compiler execution.

Q2 What is JVM, JRE, and JDK? How are they different?

Component	Full Name	Role / Contents
JVM	Java Virtual Machine	Abstract execution engine that loads, verifies, and executes Java Bytecode (`.class` files).
JRE	Java Runtime Environment	JVM + Core Class Libraries (`.rt.jar`). Environment required to <i>*run*</i> Java applications.
JDK	Java Development Kit	JRE + Development Tools (`.javac` compiler, `.javadoc`, `.jar`). Tool kit required to <i>*build & run*</i> Java applications.

Q3 What is bytecode? How does Java achieve platform independence?

Bytecode is the intermediate, platform-neutral machine code generated by the Java Compiler (`javac`) saved in `.class` files.

Platform Independence: Source code is compiled once into bytecode. Any operating system (Windows, Linux, macOS) with a compatible JVM installed can execute this identical bytecode by translating it into native host machine instructions.

Q4 Difference between JIT compiler and interpreter

- **Interpreter:** Translates and executes bytecode line-by-line. Fast startup time, but slower repetitive execution.
- **JIT (Just-In-Time) Compiler:** Compiles frequently executed bytecode chunks ("hot spots") directly into native machine code at runtime and caches it, dramatically boosting execution speed.

Q5 Data types in Java (Primitive vs Non-Primitive)

- **Primitive Types (8):** Value types stored directly on Stack memory: `byte` , `short` , `int` , `long` , `float` , `double` , `char` , `boolean` .
- **Non-Primitive Types (Reference Types):** Objects stored on Heap memory referenced by pointers: Classes, Interfaces, Arrays, Enums, Strings.

Q6 Default values of primitive data types

Type	Default Value	Size
<code>byte</code> , <code>short</code> , <code>int</code>	<code>0</code>	8-bit / 16-bit / 32-bit
<code>long</code>	<code>0L</code>	64-bit
<code>float</code> / <code>double</code>	<code>0.0f</code> / <code>0.0d</code>	32-bit / 64-bit
<code>char</code>	<code>''</code> (Null character)	16-bit Unicode
<code>boolean</code>	<code>false</code>	1-bit (logical)
Object Reference	<code>null</code>	Reference pointer

Q7 Type Casting (Implicit vs Explicit)

- **Implicit Casting (Widening):** Converting smaller type to larger type automatically by Java (e.g. `int` → `double`). Safe, no data loss.
- **Explicit Casting (Narrowing):** Manual conversion from larger type to smaller type (e.g. `int x = (int) 3.14`). Risk of data truncation or precision loss.

Q8 Difference between `==` and `.equals()`

Comparison	<code>==</code> Operator	<code>.equals()</code> Method
Check Type	Reference Comparison (Memory address identity).	Value Comparison (Logical content equality).
Primitives	Compares actual primitive values.	Cannot be used on primitives.
Objects	Checks if both pointers reference the exact same memory instance.	Overridden by <code>String</code> , <code>Integer</code> , etc., to evaluate content.

Q9 `int` vs `Integer` & Autoboxing/Unboxing

`int` is a primitive value type. `Integer` is an Object wrapper class.

- **Autoboxing:** Automatic conversion from primitive to wrapper (e.g. `Integer obj = 10;`).
- **Unboxing:** Automatic conversion from wrapper to primitive (e.g. `int val = obj;`).

Q10 Variables & Types (Local, Instance, Static)

- **Local Variable:** Declared inside a method/block. Scope restricted to method. Stored on Stack. No default value.
- **Instance Variable:** Declared inside class outside methods. Bound to object instances. Stored on Heap. Automatically assigned default values.
- **Static Variable:** Declared with `static` keyword. Shared globally across all class instances. Stored in Metaspace/Heap.

2. OOP Basics in Java (Q11 - Q20)

Q11 What is a Class and an Object in Java?

A **Class** is a blueprint/template defining attributes (fields) and behaviors (methods). An **Object** is a physical memory instance of a class allocated on the Heap via the `new` keyword.

Q12 Constructor vs Method

Feature	Constructor	Method
Purpose	Initializes newly created object instances.	Defines executable object behaviors.
Name	Must match class name exactly.	Can be any arbitrary identifier.
Return Type	No return type (not even <code>void</code>).	Must specify a return type or <code>void</code> .
Invocation	Called automatically during <code>new</code> instantiation.	Called manually via object reference.

Q13 Method Overloading & Overloading `main()`

Method Overloading: Defining multiple methods in the same class with identical names but different parameter lists (Compile-time polymorphism).

Overloading `main()`: Yes! You can overload `main()`, but the JVM will ONLY invoke `public static void main(String[] args)` as the application entry point.

Q14 Method Overriding & Rules

Method Overriding: Redefining a parent class method in a subclass with exact same signature (Runtime polymorphism).

Rules: Must have exact same method name and parameters, return type must be identical or covariant, access modifier cannot be more restrictive, and cannot override `private`, `static`, or `final` methods.

Q15 What is the `super` keyword used for?

1. Invoking parent class constructor: `super()` or `super(args)` .
2. Accessing hidden parent instance fields: `super.fieldName` .
3. Invoking parent overridden methods: `super.methodName()` .

Q16 What is the `this` keyword used for?

Refers to the current executing object instance.

- Resolving parameter shadowing: `this.name = name;`
- Constructor chaining: `this(arg1, arg2);`
- Method chaining: `return this;`

Q17 Abstract Class vs Interface

Feature	Abstract Class	Interface
Methods	Can have concrete and abstract methods.	Abstract by default. Modern Java (8+) allows <code>default</code> and <code>static</code> methods.
Fields	Can have instance variables, non-final fields.	Only <code>public static final</code> constants.
Inheritance	Single class inheritance (<code>extends</code>).	Multiple interface implementation (<code>implements</code>).

Q18 Can an interface have method implementations?

Yes (since Java 8)!

- **Default Methods** (`default void log() { ... }`): Allows adding new methods to interfaces with default implementation without breaking existing implementors.
- **Static Methods** (`static void check() { ... }`): Utility methods belonging to the interface.
- **Private Methods (Java 9+)**: Helper methods internal to default methods inside the interface.

Q19 What is encapsulation? How to achieve it?

Encapsulation wraps data (fields) and methods into a single unit (class) and hides internal state from direct external access.

How to achieve: Declare instance variables `private` and provide public `getter` and `setter` methods containing validation logic.

Q20 Access Modifiers Scope

Modifier	Same Class	Same Package	Subclass (Different Pkg)	Outside World
<code>public</code>	Yes	Yes	Yes	Yes
<code>protected</code>	Yes	Yes	Yes	No
<code>default</code> (package-private)	Yes	Yes	No	No
<code>private</code>	Yes	No	No	No

abc 3. Strings (Q21 - Q26)

Q21 Why is String immutable in Java?

1. **String Constant Pool (SCP):** Reuses string literals to save memory. Immutability guarantees shared references cannot be mutated by one client affecting others.
2. **Security:** Strings store sensitive data (DB passwords, URLs). Immutability prevents tampering.
3. **Thread Safety:** Immutability makes String instances implicitly thread-safe without synchronization.
4. **HashCode Caching:** `hashCode()` is calculated once and cached, speeding up HashMap key lookups.

Q22 String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread Safety	Thread-safe (Immutable)	NOT Thread-safe	Thread-safe (Synchronized methods)
Performance	Slow on string concatenation	Fastest (No sync overhead)	Slower than StringBuilder

Q23 What is the String Constant Pool (SCP)?

The **String Constant Pool** is a special memory area inside the Heap where String literals are cached. When creating a literal (`String s = "hello"`), JVM checks SCP first. If present, it reuses the reference; if absent, it creates a new object in SCP.

Q24 `String s = "abc"` vs `String s = new String("abc")`

- `String s = "abc"` : Creates at most **1 object** in the String Constant Pool (reuses reference if already present).
- `String s = new String("abc")` : Always creates **2 objects** (1 on general Heap memory, and 1 in SCP if "abc" was not already present).

Q25 How to compare strings & why not use `==`?

Use `s1.equals(s2)` or `s1.equalsIgnoreCase(s2)` .

Why not `==` : `==` compares memory address references, not character content. Comparing `"abc" == new String("abc")` evaluates to `false` because they point to different memory addresses!

Q26 StringBuilder vs StringBuffer Thread Safety

`StringBuffer` methods are marked with `synchronized` keywords, protecting shared data against multithreaded race conditions. `StringBuilder` omits synchronization, making it significantly faster for single-threaded string manipulations.

⚠ 4. Exception Handling (Q27 - Q34)

Q27 What is Exception Handling? `try`, `catch`, `finally`

Exception Handling prevents application crashes by catching runtime errors and taking corrective actions.

- **try** : Contains code that might throw an exception.
- **catch** : Catches and handles specific exception types.
- **finally** : Block executed unconditionally for resource cleanup.

Q28 Checked vs Unchecked Exceptions

Property	Checked Exceptions	Unchecked Exceptions
Check Time	Checked by compiler at compile-time .	Occurs at runtime .
Hierarchy	Subclasses of Exception (excluding RuntimeException).	Subclasses of RuntimeException .
Examples	IOException , SQLException , ClassNotFoundException	NullPointerException , ArithmeticException , ArrayIndexOutOfBoundsException

Q29 `throw` vs `throws`

- **throw** : Keyword inside method body used to explicitly instantiate and throw an exception object (e.g. `throw new IllegalArgumentException();`).
- **throws** : Clause in method declaration informing callers that the method might throw specified checked exceptions (e.g. `void read() throws IOException`).

Q30 Error vs Exception

- **Error** (**java.lang.Error**): Severe system-level abnormal conditions (e.g., **OutOfMemoryError** , **StackOverflowError**). Applications should not attempt to catch.
- **Exception** (**java.lang.Exception**): Recoverable application conditions that can be caught and handled.

Q31 Can we have multiple catch blocks?

Yes! Order catch blocks from **most specific subclass to most general superclass**. Placing a parent `Exception` catch block before a child `IOException` catch block causes a compile error (unreachable code).

Q32 Does `finally` block always execute?

Yes, except under rare JVM termination scenarios:

1. Explicit invocation of `System.exit(0)`.
2. JVM crash or fatal OS process termination.
3. Hardware power failure.

Q33 What is try-with-resources? Why introduced?

Introduced in Java 7 to automatically close resources implementing `AutoCloseable` (e.g. streams, DB connections) upon block completion, eliminating verbose manual `finally` close blocks.

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    System.out.println(br.readLine());
} // Auto-closed automatically!
```

Q34 How to create a Custom Exception?

Extend `Exception` (for checked custom exceptions) or `RuntimeException` (for unchecked custom exceptions):

```
public class InvalidAgeException extends RuntimeException {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

5. Arrays and Loops (Q35 - Q39)

Q35 Array vs Collection

Feature	Array	Collection
Sizing	Fixed capacity set at creation.	Dynamic resizable growth.
Types	Holds primitives and Objects.	Holds Objects only (Generics).
Utility	Built-in indexing operator (<code>arr[0]</code>).	Rich algorithm methods (sort, search, filter).

Q36 `for`, `while`, and `do-while` loops

- `for` : Best when iteration count is known in advance.
- `while` : Pre-condition check loop. Executes 0 or more times.
- `do-while` : Post-condition check loop. Guarantees execution at least **ONCE**.

Q37 Enhanced for-loop (for-each) & when NOT to use

Concise syntax for iterating arrays/collections (e.g. `for (String s : list)`).

When NOT to use: (1) When you need to modify elements during traversal, (2) When you need element index positions, or (3) When iterating backwards.

Q38 Can arrays store different data types?

Primitive arrays hold single uniform types only (e.g. `int[]`). However, an `Object[]` array can store different object subtypes due to polymorphism (e.g. storing `String` , `Integer` , `Double` references inside `Object[]`).

Q39 Multidimensional Arrays

An array of arrays. Declared as `int[][] matrix = new int[3][3];`

⚡ 6. Java 8+ Basics (Q40 - Q44)

Q40 Lambda Expressions in Java

Anonymous function syntax (`(parameters) -> body`) introduced in Java 8 to support functional programming, eliminate boilerplate inner classes, and power Stream API pipelines.

Q41 Functional Interface Examples

An interface containing exactly **one abstract method** (annotated with `@FunctionalInterface`).

Examples: `Runnable` (`run()`), `Comparator` (`compare()`), `Callable` (`call()`), `Predicate` (`test()`), `Function` (`apply()`).

Q42 Stream API & Common Operations

Provides functional sequence transformations over collections.

```
List<String> names = List.of("Alice", "Bob", "Andrew");
List<String> result = names.stream()
    .filter(s -> s.startsWith("A"))
    .map(String::toUpperCase)
    .collect(Collectors.toList()); // ["ALICE", "ANDREW"]
```

Q43 `Optional` vs Returning `null`

`Optional<T>` is a container object explicitly modeling value presence or absence. Forces callers to handle missing values (via `orElse()` / `ifPresent()`), preventing silent `NullPointerException` crashes.

Q44 Method References in Java

Shorthand syntax for lambdas calling existing methods (`ClassName::methodName`).

- Static method: `Math::max`
- Instance method: `String::toUpperCase`
- Constructor reference: `ArrayList::new`

7. Memory & Runtime Concepts (Q45 - Q52)

Q45 Heap vs Stack Memory in Java

Memory Area	Scope & Lifespan	Stored Data
Stack Memory	Thread-private. Created/destroyed per method call frame.	Primitive local variables and object reference pointers.
Heap Memory	Shared application-wide. Managed by Garbage Collector.	Actual object instances and class member state.

Q46 Garbage Collection (GC) in Java

Automatic background process in JVM that identifies and reclaims heap memory occupied by unreachable objects (objects with zero active reference pointers).

Uses Generational Collection (Eden, Survivor, Tenured spaces) and Mark-and-Sweep algorithms.

Q47 `final` vs `finally` vs `finalize()`

Keyword / Method	Type	Role
<code>final</code>	Access Modifier	Makes variables constant, methods un-overridable, classes un-inheritable.
<code>finally</code>	Exception Block	Guarantees cleanup code execution after try-catch.
<code>finalize()</code>	Object Method	Protected method called by GC prior to object reclamation (Deprecated).

Q48 What is a Static Block? When executed?

A block initialized with `static { ... }`. Executed automatically **ONCE** when the class is loaded into JVM memory by the ClassLoader, prior to object construction or main method execution.

Q49 Static vs Instance Methods

- **Static Method:** Belongs to class. Invoked via `ClassName.method()` without creating objects. Cannot access non-static instance fields or `this`.
- **Instance Method:** Belongs to object instance. Invoked via object reference and accesses instance variables and `this`.

Q50 `public static void main(String[] args)` Signature Breakdown

- `public` : Access modifier allowing JVM launcher to invoke entry point from anywhere.
- `static` : Allows JVM to call method without creating an instance of the main class.
- `void` : Method returns no value to JVM.
- `main` : Standard entry point method name recognized by JVM.
- `String[] args` : Accepts command line arguments passed as string array.